ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO TABULAR

CO5177 · Nền tảng lập trình cho phân tích và trực quan dữ liệu

Học kỳ 261 · Năm học 2026–2027

Phân loại thu nhập trên UCI Adult

Nhóm CTG

Huỳnh Thành Đạt	2570161 · 50%
Lương Minh Duy	2570083 · 50%
Giảng viên	Lê Thành Sách

Thành phố Hồ Chí Minh, tháng 11 năm 2026

Tóm tắt

Thực nghiệm dự đoán thu nhập trên 50.000 USD mỗi năm từ 14 thuộc tính trong UCI Adult. Sau khi loại các bản ghi trùng và nhóm có nhãn mâu thuẫn, dữ liệu được chia thành ba tập. Bốn cấu hình so sánh gồm dự đoán lớp đa số, logistic regression thường, logistic regression cân bằng lớp và histogram gradient boosting.

Histogram gradient boosting đạt độ chính xác 87,48% và macro-F1 0,8185 trên 7.317 mẫu test. Trọng số cân bằng lớp làm tăng recall của logistic regression đối với nhóm thu nhập cao, đồng thời tăng số dự đoán dương tính giả.

Tổng quan báo cáo

Dữ liệu	UCI Adult với 14 thuộc tính số và phân loại.
Tiền xử lý	Loại trùng trước khi chia tập, điền giá trị thiếu và mã hóa thuộc tính phân loại.
Đối chứng	Dự đoán lớp đa số, logistic regression và histogram gradient boosting.
Kết quả chính	Histogram gradient boosting đạt độ chính xác 87,48% và macro-F1 0,8185 trên 7.317 mẫu test.

Mục lục

Tóm tắt **i**

Tổng quan báo cáo **ii**

1 Bài toán và dữ liệu **1**

1.1 Mục tiêu 1

1.2 Nguồn và cấu trúc 1

1.3 Chất lượng dữ liệu và chia tập 1

1.4 Phân bố thuộc tính số 3

2 Phương pháp và thiết lập **4**

2.1 Chia tập và tiền xử lý 4

2.2 Các mô hình 5

2.3 Phép đo 5

3 Kết quả và phân tích **7**

3.1 So sánh mô hình 7

3.2 Ảnh hưởng của mất cân bằng lớp 7

3.3 Kết quả của histogram gradient boosting 8

4 Kết luận và giới hạn **10**

4.1 Kết luận 10

4.2 Giới hạn 10

4.3 Hướng mở rộng 10

Tài liệu tham khảo **11**

Danh sách hình vẽ

1.1 Số giá trị thiếu theo thuộc tính trong toàn bộ dữ liệu nguồn trước khi chia tập và imputation. 2

1.2 Số bản ghi theo nhãn thu nhập sau khi loại nhóm nhãn mâu thuẫn và bản ghi trùng. 2

1.3 Phân bố sáu thuộc tính số trên tập train. Capital gain và capital loss tập trung ở 0, còn số giờ làm việc tập trung quanh một tuần làm việc thông thường. Trục số bản ghi dùng thang log để thể hiện các khoảng có ít mẫu; thuộc tính giữ ở thang đo gốc. 3

3.1 Histogram gradient boosting đạt accuracy 87,48% và macro-F1 0,8185, cao nhất trong bốn mô hình trên cùng tập test. Cân bằng lớp làm accuracy của logistic regression giảm từ 85,24% xuống 81,43%; kết quả ứng với một split và một seed. 7

3.2 Confusion matrix trên test của mô hình có macro-F1 validation cao nhất; hàng là nhãn thực, cột là nhãn dự đoán. 9

Danh sách bảng

3.1 Siêu tham số C được chọn theo macro-F1 trên tập validation; các cột test là kết quả trên tập test giữ riêng. Accuracy trình bày theo phần trăm, macro-F1 theo thang 0–1. 7

Chương 1

Bài toán và dữ liệu

1.1 Mục tiêu

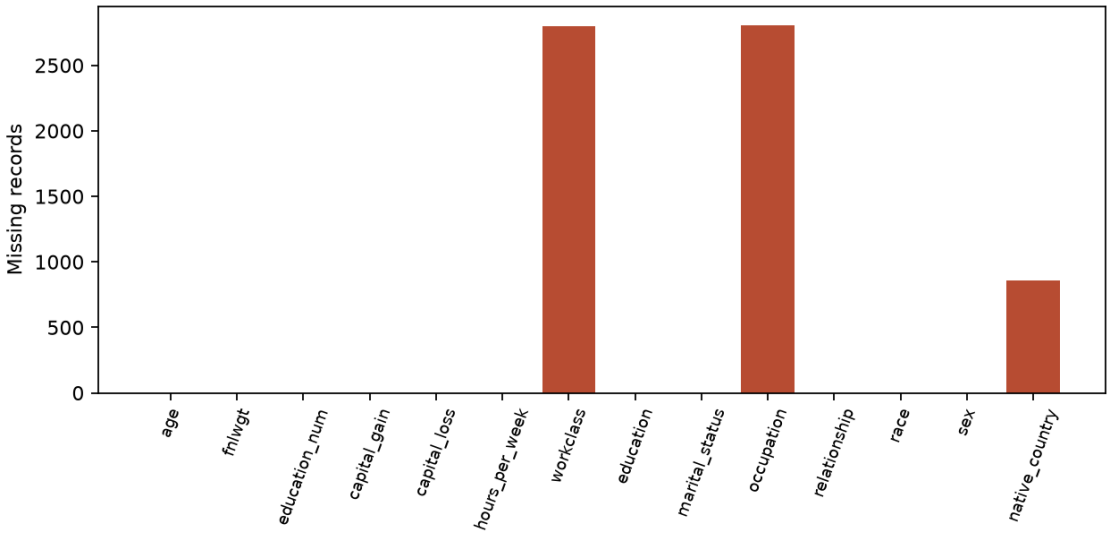
Bài toán dự đoán nhãn thu nhập lớn hơn 50.000 USD/năm từ các thuộc tính trong UCI Adult. Phần phân tích tập trung vào dữ liệu thiếu, mã hóa thuộc tính phân loại, normalization cho thuộc tính số và đánh đổi giữa accuracy với chất lượng dự đoán lớp ít mẫu. Bốn đối chứng gồm dự đoán lớp đa số, logistic regression, logistic regression cân bằng lớp và histogram gradient boosting.

1.2 Nguồn và cấu trúc

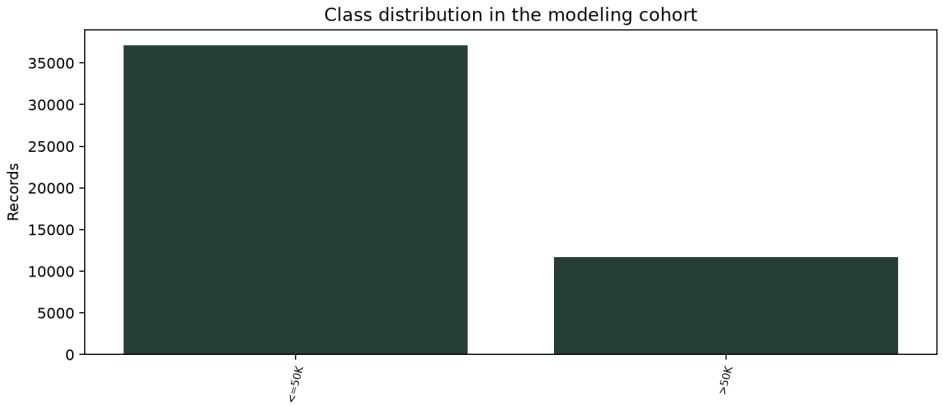
Adult gồm 48.842 bản ghi và 14 thuộc tính từ dữ liệu Census năm 1994, do Barry Becker và Ronny Kohavi cung cấp. Dữ liệu được UCI công bố [1] dưới dạng hai tệp CSV không có hàng tên cột: `adult.data` gồm 32.561 bản ghi và `adult.test` gồm 16.281 bản ghi. Mỗi dòng chứa 14 thuộc tính đầu vào và nhãn `income`. Các thuộc tính số là tuổi, fnlwgt, số năm học, capital gain, capital loss và số giờ làm mỗi tuần. Tám thuộc tính còn lại là phân loại. Dấu hỏi được đọc thành giá trị thiếu. Phân tích sử dụng các giá trị thiếu sẵn có trong dữ liệu, không tạo thiếu nhân tạo. Đích dự đoán là nhãn thu nhập, không phải giá trị thu nhập liên tục; do đó, kết quả được đánh giá như một bài toán phân loại nhị phân.

1.3 Chất lượng dữ liệu và chia tập

Hai tệp train/test gốc được gộp trước khi tạo một split mới. Nhóm có cùng 14 thuộc tính nhưng nhãn mâu thuẫn bị loại toàn bộ; nhóm trùng còn lại giữ một bản ghi. Quy tắc được áp dụng trước khi chia tập, nhằm tránh cùng một vector thuộc tính xuất hiện ở hai phía. Đây không phải phép đánh giá trên test gốc, nên điểm số không so trực tiếp với các benchmark dùng split đó. Hai bước làm sạch loại 10 dòng có nhãn mâu thuẫn và 52 dòng trùng. Thực nghiệm sử dụng toàn bộ 48.780 dòng còn lại. Các cột thiếu gồm workclass (2.799), occupation (2.809) và `native_country` (857).



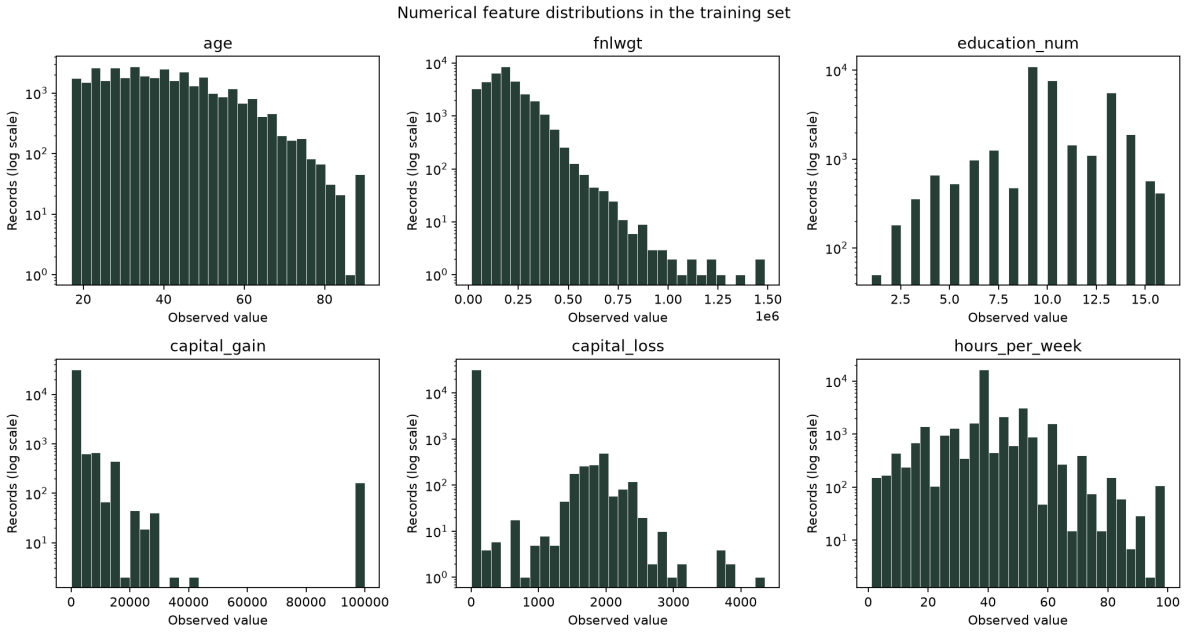
Hình 1.1: Số giá trị thiếu theo thuộc tính trong toàn bộ dữ liệu nguồn trước khi chia tập và imputation.



Hình 1.2: Số bản ghi theo nhãn thu nhập sau khi loại nhóm nhãn mâu thuẫn và bản ghi trùng.

1.4 Phân bố thuộc tính số

Khảo sát thuộc tính số chỉ sử dụng 34.146 mẫu train. Quy tắc $1{,}5\,IQR$ đánh dấu các giá trị nằm ngoài $[Q_1 - 1{,}5\,IQR;\ Q_3 + 1{,}5\,IQR]$. Cả capital gain và capital loss đều có $Q_1 = Q_3 = 0$, nên mọi giá trị dương đều bị đánh dấu, tương ứng 2.772 và 1.618 mẫu. Số giờ làm mỗi tuần có $Q_1 = 40$, $Q_3 = 45$; hai ngưỡng 32,5 và 52,5 giờ đánh dấu 9.461 mẫu. Các ngưỡng này mô tả độ tập trung và phần đuôi của phân phối, chưa đủ để xác định bản ghi sai. Vì vậy, thực nghiệm giữ nguyên các giá trị và không loại thêm mẫu theo IQR. Đặc biệt, khi $IQR = 0$, quy tắc này chỉ phân biệt giá trị 0 với giá trị dương, chứ không cho thấy mức độ bất thường của khoản thu hoặc lỗ. Với số giờ làm việc, 27,71% mẫu train bị đánh dấu; loại toàn bộ nhóm này sẽ làm mất nhiều quan sát ngoài khoảng giờ làm phổ biến. Quyết định giữ mẫu tránh đồng nhất phần đuôi phân phối với lỗi nhập liệu. Thực nghiệm chưa so sánh với clipping hay biến đổi log, nên chưa kết luận chiến lược giữ nguyên là tối ưu.



Hình 1.3: Phân bố sáu thuộc tính số trên tập train. Capital gain và capital loss tập trung ở 0, còn số giờ làm việc tập trung quanh một tuần làm việc thông thường. Trục số bản ghi dùng thang log để thể hiện các khoảng có ít mẫu; thuộc tính giữ ở thang đo gốc.

Chương 2

Phương pháp và thiết lập

2.1 Chia tập và tiền xử lý

Dữ liệu được chia phân tầng theo nhãn với tỷ lệ 70/15/15 và seed 2026; phần 30% còn lại được chia validation/test bằng seed 2027. Số mẫu train/validation/test lần lượt là 34.146/7.317/7.317. Phân tầng giữ tỷ lệ hai lớp gần nhau giữa các tập, giúp so sánh mô hình trên cùng một phân bố nhãn. Tập validation dùng để chọn cấu hình; tập test chỉ dùng để đánh giá sau khi các lựa chọn đã cố định.

Median imputer cho cột số và most-frequent imputer cho cột phân loại chỉ fit trên tập train. Cách điền này giữ lại các dòng có giá trị thiếu và tạo một quy trình chung cho các mô hình. Median ít chịu ảnh hưởng của giá trị lớn ở phần đuôi hơn trung bình, còn giá trị phân loại phổ biến nhất cung cấp một cách điền đơn giản. Tuy nhiên, việc điền có thể làm tăng tần suất nhóm phổ biến và không mô tả nguyên nhân thiếu dữ liệu. Các phép biến đổi được ghép bằng Pipeline và ColumnTransformer [2, 3], để validation và test chỉ nhận phép biến đổi đã học từ tập train.

Logistic regression dùng one-hot encoding cho thuộc tính phân loại và StandardScaler cho thuộc tính số. One-hot encoding tránh gán quan hệ thứ tự giả giữa các nhóm như nghề nghiệp hoặc tình trạng hôn nhân. Với thuộc tính số x_j, normalization có dạng

$$z_j = \frac{x_j - \mu_j}{s_j},$$

trong đó μ_j và s_j được tính trên tập train. Đưa các cột về thang đo tương đương giúp hệ số và regularization ít bị chi phối bởi khác biệt đơn vị, chẳng hạn giữa tuổi và capital gain. Phép biến đổi này không làm phân phối trở thành phân phối chuẩn và không loại giá trị cực đoan. Giá trị phân loại chưa gặp ở tập train được encoder bỏ qua khi biến đổi validation/test.

Histogram gradient boosting dùng ordinal encoding và khai báo các vị trí categorical, vì thế mã số được xử lý như mã nhóm. Mã -1 biểu diễn giá trị phân loại chưa gặp. Mô hình cây không áp dụng normalization cho thuộc tính số vì việc chia nhánh dựa trên ngưỡng, không dựa trên khoảng cách giữa các biến.

2.2 Các mô hình

Baseline luôn dự đoán lớp phổ biến nhất, cung cấp mốc so sánh để kiểm tra mức cải thiện so với việc chỉ dựa vào tỷ lệ nhãn. Hai cấu hình logistic regression dùng cùng đặc trưng và quy trình chọn siêu tham số; điểm khác nhau là trọng số lớp [4]. Với $y = 1$ ứng với thu nhập trên 50.000 USD và vector đầu vào đã tiền xử lý $\mathbf{z}$, mô hình có dạng

$$p(y = 1 \mid \mathbf{z}) = \sigma(\mathbf{w}^\top \mathbf{z} + b), \qquad \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Log-odds của mô hình là hàm tuyến tính trong không gian đặc trưng đã mã hóa; thực nghiệm không bổ sung các đặc trưng tương tác. Tham số C điều khiển nghịch đảo mức regularization: C lớn tương ứng với phạt yếu hơn. Mỗi cấu hình chọn $C \in \{0.1, 1, 10\}$ theo macro-F1 validation; khi hòa giữ ứng viên xuất hiện trước. Giới hạn optimizer là 2.000 vòng.

Với `class\_weight="balanced"`, trọng số của lớp k là

$$w_k = \frac{N}{2N_k},$$

trong đó N là số mẫu train và N_k là số mẫu thuộc lớp k trong tập này. Vì vậy, mỗi lỗi ở lớp ít mẫu đóng góp lớn hơn vào hàm mất mát. Cơ chế này thay đổi quá trình học mà không thêm hay loại mẫu; nó không bảo đảm mọi chỉ số trên validation hoặc test đều tăng.

Histogram gradient boosting kết hợp tuần tự nhiều cây để tối ưu hàm mất mát, có thể biểu diễn quan hệ phi tuyến và tương tác mà logistic regression tuyến tính không mô hình hóa trực tiếp. Cấu hình dùng 150 vòng, tối đa 31 lá mỗi cây, L2 bằng 1 và không early stopping [5]. Đây là cấu hình cố định, không phải kết quả tìm kiếm toàn diện siêu tham số. Các cấu hình đều được cố định trước khi đánh giá test và không refit trên tập train cộng validation.

2.3 Phép đo

Accuracy, macro-F1, confusion matrix và precision/recall/F1 từng lớp được tính từ dự đoán trên tập test. Với lớp thu nhập cao là lớp dương, TP là số dự đoán đúng lớp này, FP là số mẫu thu nhập thấp bị dự đoán thành thu nhập cao và FN là số mẫu thu nhập cao bị bỏ sót:

$$\text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2TP}{2TP + FP + FN}.$$

Macro-F1 là trung bình F1 của hai lớp, gán vai trò ngang nhau cho mỗi lớp dù số mẫu khác nhau. Chỉ số này được dùng để chọn C nhằm tránh việc accuracy cao che khuất khả

năng dự đoán lớp thu nhập cao. Precision và recall vẫn cần được đọc riêng vì cùng một F1 có thể đi kèm những kiểu sai khác nhau. Thực nghiệm sử dụng một split và model seed 42, chưa ước lượng độ biến thiên qua nhiều lượt chạy.

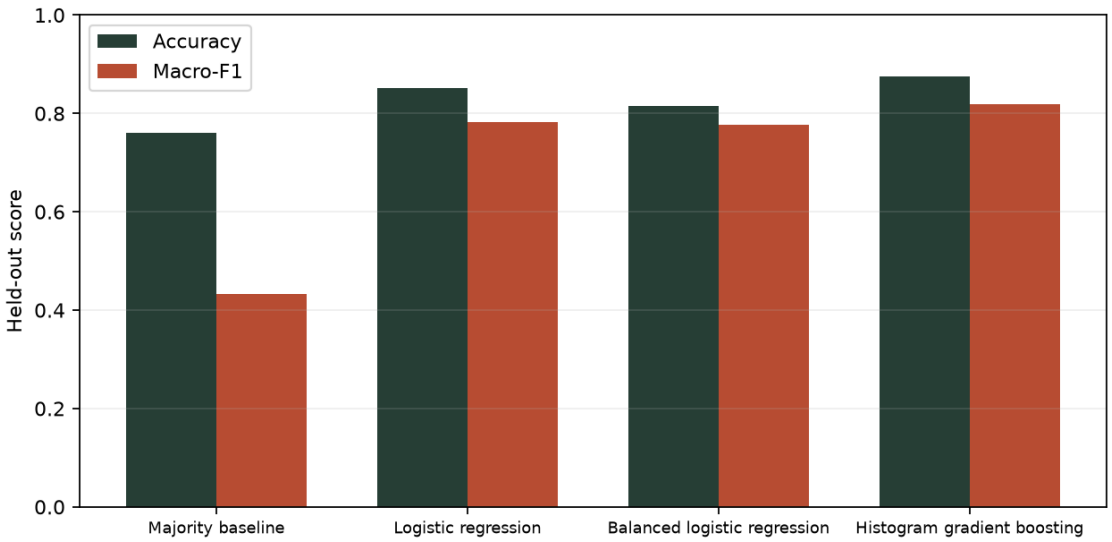
Chương 3

Kết quả và phân tích

3.1 So sánh mô hình

Mô hình	C	Val macro-F1	Test accuracy	Test macro-F1
Majority baseline	–	0.4320	76.06%	0.4320
Logistic regression	10.0	0.7803	85.24%	0.7815
Balanced logistic regression	10.0	0.7639	81.43%	0.7769
Histogram gradient boosting	–	0.8147	87.48%	0.8185

Bảng 3.1: Siêu tham số C được chọn theo macro-F1 trên tập validation; các cột test là kết quả trên tập test giữ riêng. Accuracy trình bày theo phần trăm, macro-F1 theo thang 0–1.



Hình 3.1: Histogram gradient boosting đạt accuracy 87,48% và macro-F1 0,8185, cao nhất trong bốn mô hình trên cùng tập test. Cân bằng lớp làm accuracy của logistic regression giảm từ 85,24% xuống 81,43%; kết quả ứng với một split và một seed.

3.2 Ảnh hưởng của mất cân bằng lớp

Tập test có 5.565 mẫu thu nhập không quá 50.000 USD và 1.752 mẫu thu nhập cao. Vì lớp thứ nhất chiếm 76,06%, baseline đạt accuracy 76,06% dù bỏ sót toàn bộ lớp thu nhập cao. Macro-F1 chỉ đạt 0,4320 và recall lớp thu nhập cao bằng 0. Điều này cho thấy

accuracy cần được đọc cùng các chỉ số từng lớp, thay vì dùng một mình để đánh giá chất lượng phân loại.

Logistic regression thông thường đạt accuracy 85,24% và macro-F1 0,7815, cải thiện rõ so với baseline. Tuy vậy, mô hình chỉ nhận đúng 1.035 trong 1.752 mẫu thu nhập cao và bỏ sót 717 mẫu. Khi thêm trọng số cân bằng lớp, recall của nhóm này tăng từ 0,5908 lên 0,8459, trong khi precision giảm từ 0,7403 xuống 0,5764. Số mẫu bị bỏ sót giảm từ 717 xuống 270, nhưng số dự đoán dương tính giả tăng từ 363 lên 1.089. Cụ thể, tổng số mẫu thu nhập cao được dự đoán đúng tăng 447, còn tổng số mẫu thu nhập thấp bị dự đoán sai thành thu nhập cao tăng 726.

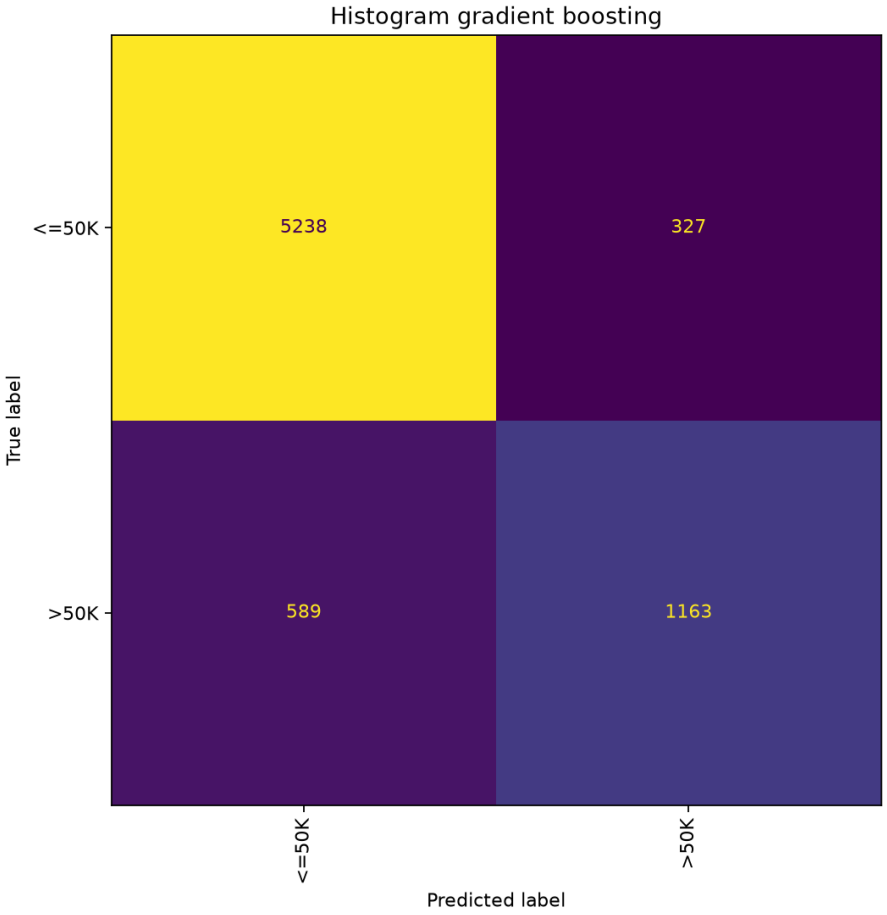
F1 của lớp thu nhập cao tăng từ 0,6571 lên 0,6856, còn F1 của lớp thu nhập thấp giảm từ 0,9060 xuống 0,8682. Do đó, macro-F1 chung giảm nhẹ từ 0,7815 xuống 0,7769 dù recall của lớp ít mẫu tăng mạnh. Cân bằng trọng số lớp là một cách ưu tiên giảm bỏ sót, nhưng không tự động tối ưu macro-F1. Nếu mục tiêu đặt chi phí cao hơn cho bỏ sót, cấu hình balanced có thể đáng cân nhắc; nếu cần hạn chế dự đoán dương tính giả, precision thấp hơn là đánh đổi đáng kể. Thực nghiệm chưa xác định chi phí nghiệp vụ hay chọn ngưỡng dự đoán riêng cho hai mục tiêu này.

3.3 Kết quả của histogram gradient boosting

Histogram gradient boosting đạt accuracy 87,48% và macro-F1 0,8185, cao hơn logistic regression thông thường 2,24 điểm phần trăm accuracy và 0,0370 macro-F1. Thứ hạng này cũng xuất hiện trên validation, nơi boosting đạt macro-F1 0,8147. Sự nhất quán giữa hai tập hỗ trợ lựa chọn boosting trong phạm vi cấu hình đã thử, nhưng chưa đủ để kết luận về độ ổn định qua các split khác.

Confusion matrix cho thấy boosting dự đoán đúng 1.163 mẫu thu nhập cao, bỏ sót 589 mẫu và tạo 327 dương tính giả. So với logistic regression thông thường, cả số bỏ sót lẫn số dương tính giả đều giảm; precision và recall của lớp thu nhập cao lần lượt đạt 0,7805 và 0,6638. Tuy nhiên, mô hình vẫn bỏ sót khoảng một phần ba lớp thu nhập cao. Vì vậy, accuracy 87,48% không đồng nghĩa với việc hai lớp được dự đoán tốt như nhau.

Khả năng biểu diễn phi tuyến và tương tác là một khác biệt về phương pháp giữa boosting và logistic regression. Tuy nhiên, hai pipeline cũng sử dụng cách mã hóa thuộc tính phân loại khác nhau; thực nghiệm chưa tách riêng các yếu tố này nên không thể quy toàn bộ mức cải thiện cho một cơ chế cụ thể. Kết quả chỉ xác định mô hình tốt nhất trong bốn cấu hình trên cùng split, chưa chứng minh boosting luôn phù hợp hơn cho mọi dữ liệu thu nhập.



Hình 3.2: Confusion matrix trên test của mô hình có macro-F1 validation cao nhất; hàng là nhãn thực, cột là nhãn dự đoán.

Chương 4

Kết luận và giới hạn

4.1 Kết luận

Trên toàn bộ dữ liệu sau làm sạch và split đã cố định, histogram gradient boosting đạt kết quả cao nhất trong các cấu hình được khảo sát. Logistic regression cân bằng lớp tăng recall của nhóm thu nhập cao nhưng giảm precision và accuracy. Vì vậy, lựa chọn trọng số lớp cần gắn với mức chấp nhận bỏ sót và dự đoán dương tính giả của bài toán.

4.2 Giới hạn

Các kết quả phản ánh một cách chia dữ liệu và một seed, chưa có khoảng tin cậy hoặc phép kiểm định cho chênh lệch giữa các mô hình. Miền tìm kiếm C còn hẹp và cấu hình boosting được cố định, nên so sánh chưa đại diện cho hiệu quả tối ưu của từng họ mô hình.

Nguồn điều tra năm 1994 giới hạn khả năng áp dụng cho thu nhập hiện nay. Dữ liệu không có mã hộ gia đình để chia tập theo hộ; loại các vector thuộc tính trùng chỉ giải quyết một dạng trùng lặp quan sát. Fnlwgt được dùng làm biến đầu vào, không làm trọng số mẫu, nên các chỉ số phản ánh các bản ghi trong tập test chứ không phải ước lượng có trọng số cho toàn bộ dân số.

Chưa khảo sát riêng ảnh hưởng của từng chiến lược imputation, xử lý giá trị cực đoan hay hiệu quả trên từng nhóm nhân khẩu. Các đặc trưng dự đoán và chênh lệch chỉ số không cung cấp bằng chứng về quan hệ nhân quả với thu nhập.

4.3 Hướng mở rộng

Các hướng tiếp theo gồm lặp lại trên nhiều split để đánh giá độ biến thiên, so sánh chiến lược imputation và xử lý phần đuôi phân phối, đồng thời chọn ngưỡng dự đoán theo chi phí bỏ sót và dương tính giả. Mỗi thay đổi cần được chọn bằng validation theo giao thức xác định trước, sau đó đánh giá trên tập test giữ riêng. Đây là các đề xuất chưa được thực nghiệm.

Phạm vi hỗ trợ của AI được trình bày trong AI_USE_REPORT.md.

Tài liệu tham khảo

[1] B. Becker và R. Kohavi. *Adult*. UCI Machine Learning Repository, 1996. DOI: 10.24432/C5XW20. https://archive.ics.uci.edu/dataset/2/adult.

[2] Scikit-learn. *Pipeline*. https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html.

[3] Scikit-learn. *ColumnTransformer*. https://scikit-learn.org/stable/modules/generated/sklearn.compose.ColumnTransformer.html.

[4] Scikit-learn. *LogisticRegression*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html.

[5] Scikit-learn. *HistGradientBoostingClassifier*. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.HistGradientBoostingClassifier.html.